{dev/talles}

Goroutines
Go channels

Por: Ricardo Cuellar

{dev/talles}

Concurrencia

Es la capacidad de manejar múltiples tareas en progreso al mismo tiempo. No implica que ocurran simultáneamente.

{dev/talles}

Paralelismo

Ejecución literal y simultánea de múltiples tareas en múltiples CPUs/cores.

{dev/talles}

Goroutines

Es el mecanismo concreto que Go usa para implementar la concurrencia.

Son funciones que se ejecutan concurrentemente. Más ligeras que threads del S.O.

{dev/talles}

Goroutines - Datos importantes

- Peso mínimo
- Multiplexadas
- Gestionadas por Go
- Stack dinámico
- Creación barata
- No hay ID

{dev/talles}

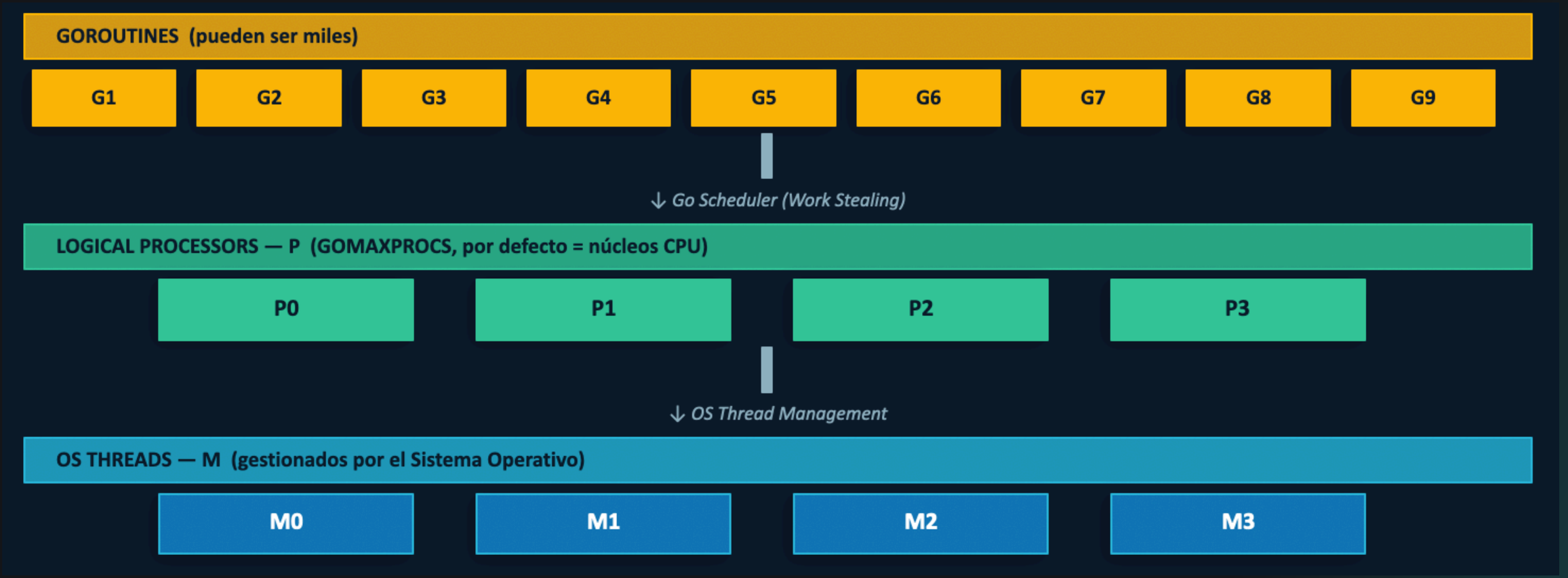
Goroutines - ¿Cómo funcionan?

goroutine.go

```go
func greet(name string){
  fmt.Println("Hola, ", name)
}


//Lanzar como goroutine
go greet("Ricardo")

//Función anónima
go func(){
  fmt.Println("Soy concurrente!")
}()

// Goroutines en loop
for i:=0; i < 1000; i++{
  go processTask(i)
}

// main() es Goroutine
```

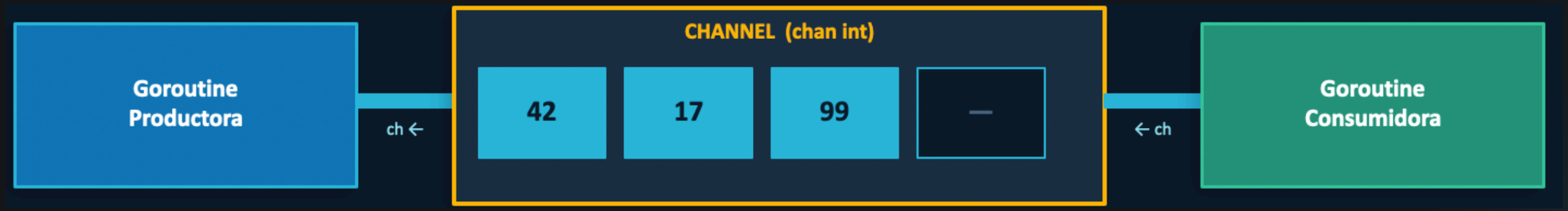
{dev/talles}

El Scheduler de Go - Modelo M:N



{dev/talles}

Go Channels - Comunicación entre Go routines



Unbuffered (sync)

El emisor bloquea hasta que alguien recibe. Sincronización garantizada.

unbuffered.go

```go
ch := make(chan int)
```

Buffered (async)

El emisor bloquea solo si el buffer está lleno. Permite cierto desacoplamiento.

buffered.go

```go
ch := make(chan int, 10)
```

Direccionales

Solo lectura **(<-chan)** o solo escritura **(chan <-)**. Mejoran seguridad en tipo.

directions.go

```go
func f(ch <-chan int) {...}
```

{dev/talles}

Herramientas esenciales del Ecosistema

{dev/talles}

select - Multiplexar Channels

select.go

```go
select {
case msg := <-ch1:
    fmt.Println("ch1:", msg)
case msg := <-ch2:
    fmt.Println("ch2:", msg)
case <-time.After(1 * time.Second):
    fmt.Println("timeout!")
default:
    fmt.Println("sin datos")
}
```

{dev/talles}

sync.WaitGroup - Esperar goroutines

wait_group.go

```go
var wg sync.WaitGroup

for i := 0; i < 5; i++ {
    wg.Add(1)
    go func(n int) {
        defer wg.Done()
        procesar(n)
    }(i)
}

wg.Wait() // Bloquea hasta que Done() = 0
```

{dev/talles}

sync.Mutex - Proteger datos compartidos

mutex.go

```
var mu sync.Mutex
contador := 0

go func() {
    mu.Lock()
    contador++
    mu.Unlock()
}()
```

{dev/talles}

context - Cancelación y timeouts

context.go

```go
ctx, cancel := context.WithTimeout(
    context.Background(), 5*time.Second)
defer cancel()

go func() {
    select {
    case <-ctx.Done():
        return // cancelado o timeout
    }
}()
```

{dev/talles}

Ventajas

- Muy ligeras. 2KB de stack inicial vs 1MB de un thread.
- Comunicación segura.
- Modelo mental simple, solo usas la palabra go.
- Alto rendimiento I/O
- Prevención de data races
- Primitivas integradas

{dev/talles}

Desventajas

- Goroutine leaks. Puedes hacer que una goroutine consuma memoria de forma indefinida.
- Data races.
- Deadlocks
- Complejidad en depuración

¿Cuándo SI usarlos?

- I/O intensivo
- Servidores web
- Pipelines de datos
- Operaciones independientes
- Fan-out / Fan-in (Distribuir trabajo de workers)
- Background jobs

{dev/talles}

¿Cuándo NO usarlos?

- Tareas puramente del CPU
- Lógica secuencial simple
- Sin sincronización clara
- Compartir mucho estado.
- Scripts corto o CLIs muy simples.
- Si no hay plan de ciclo de vida claro

{dev/talles}

GO

Gracias

www.devtalles.com

/ricardocuellar

@ricardocuellars